AI/ML Detection of Suspicious Mule Bank Accounts

A Complete Reference Document

Problem context · domain primer · solution walkthrough

Backend work to date · Plan to submission

Prepared for: Hackathon Submission

Deadline: 5 June 2026

Contents

1. Read this first — what this document gives you
2. The world of bank fraud: foundational concepts
3. What is a mule account, really?
4. The problem statement, decoded
5. Why this is hard — the AI/ML angle
6. The dataset we were given
7. What we built in the backend — phase by phase
8. The model results — what the numbers mean
9. Architecture: the system around the model
10. Where we are now and what comes next
11. Glossary of every term used

1. Read this first — what this document gives you

This document is written for someone who has not seen this problem before. It explains the banking fraud domain from scratch, walks through what the hackathon is asking for, describes what we have built so far in the backend, and lays out what remains before submission.

You do not need any prior banking or machine learning background to read this. Every term is introduced before it is used. The first half is conceptual. The second half walks through the actual code and decisions made in the project.

How to use it

- Read sections 1–5 once, slowly. These are the concepts you need to internalize to discuss this project confidently.
- Use sections 6–9 as a reference. They describe what exists in the repository today.
- Section 10 is the runway to submission day.
- Section 11 is the glossary. Every acronym and jargon term used in the project is defined there.

> **If you only read one paragraph**
>
> Criminals use innocent people's bank accounts to move stolen money. These borrowed accounts are called mule accounts. Banks need to detect them automatically before the money leaves. We are building a machine learning model that scores each account on how likely it is to be a mule, plus the system around the model that turns scores into actions like 'freeze this account' or 'route to a human analyst.' The model is built and works well. The remaining work is the user interface and demo around it.

2. The world of bank fraud: foundational concepts

Before getting into mule accounts specifically, you need a working mental model of how money moves between bank accounts and where fraud happens.

How money moves between accounts in India

When someone transfers money in India, it happens through one of several channels. The hackathon problem mentions all of these because each is a different route fraudsters use.

Channel	What it is	Typical use
UPI	Unified Payments Interface — instant transfer via phone, scan-and-pay	Everyday small payments, QR-code shops, peer-to-peer
IMPS	Immediate Payment Service — instant bank-to-bank transfer	Sending money 24×7, larger than typical UPI
NEFT	National Electronic Funds Transfer — batched bank transfer	Salary, larger transfers, business payments
RTGS	Real-Time Gross Settlement — instant transfer for large amounts	Property purchases, business settlements, ₹2 lakh and above
AePS	Aadhaar-enabled Payment System — withdraw using fingerprint at agent	Rural cash-out without a card; common in remote areas
Cards	Debit and credit card transactions, ATM withdrawals	POS purchases, online shopping, cash withdrawals
Net Banking	Logging into bank's website to transfer or pay	Bill payments, transfers from home computer
Branch / CBS	In-person at a branch, recorded in Core Banking System	Cash deposits, cheque deposits, account opening

Every one of these channels is a potential entry or exit point for fraudulent money. A modern fraud system has to watch all of them because criminals deliberately spread activity across channels to avoid pattern detection on any single one.

How a typical fraud unfolds

Cyber-enabled financial fraud in India usually follows this pattern. Understanding this sequence is essential because the mule account sits in the middle of it.

1. Initial contact — A fraudster contacts a victim by phone, SMS, WhatsApp, or a fake website. Common pretexts include fake job offers, KYC update calls pretending to be from the bank, loan offers, investment schemes, or romance scams.

2. Trust building — The fraudster establishes credibility. They may share fake bank statements, government IDs, or website links.
3. Extraction — The victim is convinced to transfer money or share OTPs and passwords. This is the moment of fraud.
4. Layering — The stolen money is immediately moved through several intermediate accounts. These intermediate accounts are mule accounts. The point is to make the money hard to trace and recover.
5. Cash-out — The money is eventually withdrawn as cash through ATMs, AePS, or converted to cryptocurrency, gold, or international transfers.
6. Investigation — The victim discovers the fraud, files a complaint with the bank and with national portals like NCRP. By this time the money is often gone.

> **The critical window**
>
> The time between Step 3 (extraction) and Step 5 (cash-out) is often less than an hour. Sometimes it is less than 10 minutes. This is why automated, real-time detection of mule accounts matters — by the time a human investigator looks at the case, the money has already moved.

Who regulates this in India

Several institutions are involved in fighting financial fraud in India. They appear repeatedly in the problem statement and in our solution design.

Acronym	Full name	What they do
RBI	Reserve Bank of India	Central bank. Issues regulations all banks must follow.
FIU-IND	Financial Intelligence Unit, India	Collects suspicious transaction reports from banks. Coordinates with police.
I4C	Indian Cyber Crime Coordination Centre	Federal cyber-crime coordination body.
NCRP	National Cybercrime Reporting Portal	Citizen-facing website where fraud victims file complaints.
CFCFRMS	Citizen Financial Cyber Fraud Reporting and Management System	I4C's backend system that routes reported fraud to banks for action.
STR	Suspicious Transaction Report	Formal report a bank files with FIU-IND when it spots suspicious activity.
KYC	Know Your Customer	Identity verification a bank does before opening an account.

Acronym	Full name	What they do
CDD	Customer Due Diligence	Ongoing verification of customer activity, deeper than KYC.

3. What is a mule account, really?

A mule account is a bank account that is used to receive and forward illicit funds, where the account holder is either an unwitting participant or a paid accomplice. The term comes from drug trafficking, where a 'mule' is a person who carries contraband. In financial fraud, the mule is the account itself.

Three flavours of mule

Type	How they came to be a mule	Behavioural signature
Unwitting	Tricked into sharing account access — 'I'll deposit your salary, just share OTP'	Account was dormant or low-activity, suddenly receives and forwards large sums
Willing for payment	Sold or rented out their account knowingly. Common among students, low-income workers	Account opened recently, KYC valid, but used immediately for high-velocity transfers
Synthetic / fully fraudulent	Account opened with fake or stolen identity documents specifically for fraud	Brand-new account, minimal real activity, used purely as a pass-through

The behavioural fingerprint of a mule

Despite the three types looking different on paper, in transaction data they share a recognizable pattern. Our model is trying to learn this fingerprint. The pattern includes:

- **Pass-through behaviour:** Money comes in and leaves quickly. The account balance does not grow. Mule accounts are conveyor belts, not safes.
- **Fan-in and fan-out:** Receives from many different sources, sends to many different destinations. Legitimate personal accounts usually transact with a small consistent set of counterparties.
- **Channel diversity:** Uses multiple channels (UPI, IMPS, ATM) in short windows. Most real users have one or two preferred channels.
- **Dormancy break:** An account quiet for months suddenly becomes very active. Classic for accounts that have been bought or stolen.
- **Structuring:** Transactions just below reporting thresholds (₹49,999 to stay under the ₹50,000 trigger; ₹1,99,999 to stay under ₹2 lakh). Deliberate evasion.
- **Device and IP sharing:** Multiple supposedly-different accounts logging in from the same phone or IP address.
- **Geographic mismatch:** KYC address in one state, transactions originating from another — especially common in mule networks operated by gangs.

Why traditional rules fail

Banks have rule-based fraud detection systems that look for specific patterns: 'more than X transactions in Y hours', 'transfer above ₹Z to a new beneficiary', and so on. These rules catch a lot, but they have two problems.

First, fraudsters learn the thresholds and stay just under them. Structuring is the obvious example, but it applies to every rule. If the rule fires above ₹50,000, they send ₹49,500. If it fires after 10 transactions per hour, they do 9 per hour from one account and use another.

Second, rules cannot capture interactions between many small signals at once. A single transaction at 3 AM is not suspicious. A new beneficiary is not suspicious. A round-number amount is not suspicious. But all three together, in a previously-dormant account, very probably is suspicious. Machine learning models learn these combined patterns from data.

This is what the hackathon is asking for

Build a machine learning system that learns these complex behavioural patterns from data, rather than relying on hand-coded rules. The system should be able to identify mule accounts with high accuracy, explain why it flagged them, integrate with regulatory feeds, and run in real time.

4. The problem statement, decoded

The hackathon gives a problem statement that contains a lot of dense language. Here it is unpacked line by line.

The official statement

> **From the hackathon brief**
>
> Develop a solution having AI/ML capabilities for detecting suspicious transactions and mule accounts by ingesting financial transactions and/or fraud monitoring solution alerts and/or Transaction monitoring system alerts and govt cyber fraud alerts/tickets and preventing circulation of fraudulent proceeds through mule accounts. This solution should consume real-time regulatory inputs/feeds and cross-channel bank data.

What each phrase actually means

'AI/ML capabilities for detecting suspicious transactions and mule accounts'

Build a machine learning model that scores each account on its likelihood of being a mule. Not a rule-based system. The model must learn patterns from data rather than being told them by humans.

'Ingesting financial transactions'

The system has to consume the raw transaction stream from the bank — every UPI payment, every NEFT transfer, every ATM withdrawal across all the channels listed earlier. In production this is thousands or millions of transactions per day, arriving in real time.

'And/or fraud monitoring solution alerts and Transaction monitoring system alerts'

FMS (Fraud Monitoring System) and TMS (Transaction Monitoring System) are existing software the bank already runs. They use simple rules to flag transactions. Our system should ingest their alerts as additional input — treating these alerts as features that feed our model, not as final answers.

'And govt cyber fraud alerts/tickets'

When a citizen reports a fraud on NCRP, the I4C system routes that complaint to the bank where the suspect account is held. These tickets identify specific account numbers or UPI IDs that have been named in fraud complaints. Our system has to consume this feed and treat named accounts as high-confidence positives — they get blocked immediately, not just scored.

'Preventing circulation of fraudulent proceeds through mule accounts'

Detection alone is not enough. The system has to actually stop money from moving — by freezing accounts, holding outgoing transfers, requiring step-up authentication, or routing transactions to a human analyst before they complete.

'Should consume real-time regulatory inputs/feeds and cross-channel bank data'

Two requirements. First, the system has to be plugged into live data feeds from RBI, FIU-IND, and I4C, not pull them once a day. Second, it has to see all the bank's channels together — looking at UPI alone or cards alone misses mule patterns that show up across channels.

What we have to deliver

Translating the problem statement into concrete deliverables for the hackathon:

7. A trained classification model that takes an account's behavioural features and outputs a risk score.
8. Evidence that the model works: precision, recall, calibration, evaluated honestly on held-out data.
9. Explainability: for each flagged account, the model has to say why it was flagged.
10. A system architecture that shows how the model would work in a real bank — ingesting streams, scoring, deciding actions, logging.
11. A working demo that judges can interact with.
12. A report and presentation that communicates all of the above.

5. Why this is hard — the AI/ML angle

Mule account detection is one of the harder applied ML problems for several specific reasons. Understanding why it is hard makes our design choices later make sense.

Difficulty 1 — Extreme class imbalance

In our dataset, out of 9,082 accounts, only 81 are confirmed mules. That is 0.89%. In the real world the rate is even lower.

This causes practical problems. A naive model can achieve 99.1% accuracy by predicting 'not a mule' for every single account. The accuracy looks great but the model is useless. So accuracy is the wrong metric. We have to use metrics designed for imbalanced data — precision, recall, PR-AUC — explained in section 8.

Worse, with only 81 positive examples to learn from, the model has very little signal. Any single feature can look important by accident. We have to use techniques like stability selection across multiple data splits to filter out features that only look useful by chance.

Difficulty 2 — Adversarial behaviour

Fraudsters are actively trying to look normal. Unlike, say, image classification where a cat does not try to look like a dog, mule operators are deliberately mimicking legitimate account behaviour. This means the model has to find subtle multivariate patterns rather than obvious single-feature giveaways.

Difficulty 3 — Concept drift

The patterns mules use change over time. A model trained on last year's mules may fail on this year's mules. The system has to be retrainable and has to monitor itself for drift — flagging when the incoming data starts to look different from what the model was trained on.

Difficulty 4 — Cost asymmetry

Missing a mule (a false negative) means fraud proceeds escape. Flagging a legitimate account as a mule (a false positive) means a customer's transaction is blocked or their account is frozen for no reason. These two errors have very different costs and the model has to be tuned with that in mind.

This is also why the system uses risk bands rather than a single yes/no decision. Borderline cases get routed to human analysts instead of getting auto-blocked.

Difficulty 5 — Anonymized features

The hackathon dataset has 3,924 features named F1 through F3924. We do not know what most of them mean. Only 18 are 'hinted' by the bank as commonly useful. This is realistic — in real banking projects,

feature semantics are often partially obscured for privacy and competitive reasons. But it means we have to find patterns without domain context and we have to be especially careful about leakage (when a feature accidentally contains the answer).

Difficulty 6 — Explainability requirement

In regulated industries, you cannot just say 'the model said so.' If the bank freezes a customer's account, the customer has the right to know why, regulators have the right to audit, and a human analyst has to be able to review the decision. Our model uses SHAP values (explained later) to produce per-account explanations for every prediction.

6. The dataset we were given

Shape and basics

The hackathon provided a CSV file with 9,082 rows and 3,925 columns. Each row is one bank account. The last column, F3924, is the target — 1 means the account is a confirmed mule, 0 means it is not.

Property	Value
Total rows (accounts)	9,082
Total columns	3,925 (including target)
Features	3,924 (F1 through F3924-but-target)
Target	F3924 — binary, 1 = mule, 0 = not mule
Positive count (mules)	81 (0.89%)
Negative count	9,001 (99.11%)
Numeric features	3,917
Object (text) features	8
Bank-hinted useful features	18 specific ones provided

The 18 bank-hinted features

The hackathon brief lists 18 specific features the bank says are commonly useful for fraud detection. We use these as a seed set to build an 'honest floor' baseline model. Their identifiers:

F115, F321, F527, F531, F670, F1692, F2082, F2122, F2582, F2678, F2737, F2956, F3043, F3836, F3887, F3889, F3891, F3894

We do not know what each one means specifically — they are anonymized. But knowing that the bank's experts found them useful gives us a defensible starting point and a way to detect leakage (if the full-feature model is much better than a model on just these 18, we need to investigate why).

Data quality findings

We ran a thorough audit of the dataset before any modelling. The findings shaped every preprocessing decision.

Finding	Count	What we did
Zero-variance columns (same value for all rows)	359	Dropped — they carry no information

Finding	Count	What we did
Columns more than 80% missing	830	Dropped — too sparse to use
Duplicate column groups (different names, same values)	434 groups covering 484 cols	Deduplicated — kept one representative per group
F2230 (snapshot month) perfectly predicts the label	100% leakage	Hard-excluded from features
L7D/L14D/L90D time windows have zero positives	680 rows	Routed to unsupervised anomaly side-channel only

The F2230 leakage story

F2230 records the snapshot month — Sep25, Oct25, Nov25, or Dec25. We discovered that all 81 mule labels are on non-October rows, and all 8,321 October rows are non-mules. F2230 was perfectly predicting the label. This is a labelling artifact: the bank gave us an October snapshot, then appended confirmed-mule rows from investigations completed in adjacent months. If we had used F2230 as a feature, the model would have learned to just check the month — and would have failed completely in production. We caught it by running a leakage audit before any modelling. This is the most important methodological finding of the project so far.

What the final supervised pool looks like

After all dedup, drops, and filtering for usable time windows:

- Features (after dedup): 2,249
- Supervised rows: 8,402
- Positives: 81 (0.96% prevalence in the supervised pool)
- Train/test split: stratified 70/30 (preserves prevalence in both)
- Train: 5,881 rows with 57 positives
- Test: 2,521 rows with 24 positives

7. What we built in the backend — phase by phase

This section walks through every piece of code that exists in the repository, what it does, and why. The goal is that anyone reading this — including you a week from now — can navigate the project end-to-end.

Repository layout

The project is organized into clearly-separated concerns. Each directory has a single responsibility.

Path	Purpose
data/raw/	The original DataSet.csv — never modified
data/processed/	Cleaned parquet files, dedup mappings, account explanations
data/feedback/	Analyst dispositions (false-positive marks, confirmations)
src/	All ML code — preprocess, features, models, evaluation, explainability
models/	Trained model pickles, version-tagged in registry.json
reports/	Markdown reports auto-generated by the pipeline
reports/figures/	PNG charts (SHAP plots, reliability diagram, score distribution)
service/	FastAPI scoring service (in progress)
stream/	Snapshot replay producer (in progress)
ui/	Next.js analyst console (in progress)
logs/	Append-only decisions log (the WORM audit trail)

Phase 1 — Exploratory Data Analysis

Before any modelling, we explored the dataset to understand its shape and surface problems. Key outputs: class balance plot, missingness heatmap, variance scan, target-rate breakdown by snapshot month and time window. This is where we discovered the F2230 leakage and the duplicate columns.

Phase 2 — Preprocessing (src/preprocess.py)

This module is the foundation of everything downstream. It runs once on the raw CSV and produces the cleaned dataset that all subsequent steps use. The order of operations matters.

13. Drop the Unnamed:0 index column from the CSV
14. Drop 359 zero-variance columns (columns where every row has the same value)

15. Drop 830 columns with more than 80% missing values
16. Deduplicate columns — hash every column's content, group identical ones, keep one representative per group. The dedup mapping is saved so we can trace any kept column back to its duplicates
17. Process the 8 object (text) columns — date columns become numeric epochs, low-cardinality categoricals get frequency encoding, F2230 is preserved as a split key but excluded from features
18. Route rows with zero-positive time windows (L7D/L14D/L90D) to a separate anomaly pool — they cannot be used for supervised classification
19. Stratified 70/30 train/test split on the target, preserving class proportions in both
20. Run the two-step leakage audit: check correlation of every remaining feature with F2230 and with the target directly. Any feature too close to either is dropped
21. Save processed parquet files and a JSON dedup map

Phase 3 — Feature engineering and selection (src/features.py)

With 2,249 features surviving preprocessing but only 57 training positives, we cannot use all features — the model would overfit immediately. We need to select a much smaller set of features that genuinely carry signal.

The selection happens in three stages:

- **Univariate filter:** For each feature, compute its ROC-AUC against the target on the training set alone. Keep the top 500 by AUC. This is a coarse filter to throw out obvious noise.
- **Bootstrap stability selection:** Re-run the filter 50 times on random 80% subsamples of the training data. A feature 'survives' only if it appears in the top 500 in at least 30 of the 50 bootstraps. This is much more conservative than running selection once — it ensures the features we keep are robustly informative, not just lucky on one slice.
- **Embedded gain check:** Train a LightGBM on the surviving features and drop any with zero gain (the model never used them to make a split). Final count: 385 features.

Output: reports/feature_selection.md documents which features survived, their bootstrap survival rate, and their univariate AUC.

Phase 4 — Modelling (src/models.py)

We train and compare several models. Each has a specific role.

Model	Role	What it tells us
L1 Logistic Regression on 18 bank-hinted features	Interpretable floor	Simplest possible model, transparent coefficients

Model	Role	What it tells us
LightGBM on 18 bank-hinted features	Leakage benchmark	Same algorithm as primary model but limited features — used to detect if the full model is winning because of leakage
LightGBM on top-20 univariate features	Cross-check	Tests whether univariate filtering alone is sufficient (it isn't)
LightGBM on 385 stable features	Primary model	The one we ship — uses all selected features
Isolation Forest	Anomaly side-channel	Unsupervised — used for the 680 rows in zero-positive time windows

Models use scale_pos_weight to handle class imbalance — telling the algorithm to weight the rare positive class roughly 111× more than the majority class. We deliberately do not use SMOTE (synthetic minority oversampling) because with only 57 training positives, synthetic data points would be statistical fiction.

The leakage audit fired automatically because the full-feature model is much better than the seed-18 model (gap of 0.51 PR-AUC). We then investigated: did the gap come from real signal in the 367 additional features, or from a hidden leakage path? We confirmed the gap is honest — feature importances showed a monotonic decline (no single feature dominating), and the top features had near-zero correlation with the leakage indicators we audited for. Documented in reports/model_results.md section 2.

Phase 5 — Evaluation and calibration

Three components, each with its own module.

src/cv_variance.py — variance estimation

Run the primary model's training procedure across 5 folds × 3 random seeds = 15 fits. Report mean ± std for the headline metrics. This is the variance estimate that tells us how stable our reported numbers are. Result: 0.781 ± 0.087 PR-AUC, which means the model is consistently strong across different data splits.

src/calibrate.py — probability calibration

Tree models like LightGBM output ranking scores, not true probabilities. A raw output of 0.8 does not mean 80% chance of being a mule. We fit an isotonic regression wrapper using internal 5-fold cross-validation (so no test data leaks into the calibration step). The result: a model whose probability outputs can be trusted as actual probabilities. Brier score on the test set: 0.0024, which is excellent calibration quality.

src/explain.py — SHAP explanations

SHAP (SHapley Additive exPlanations) assigns each feature a contribution to each prediction. For account X, the model says 'risk score 0.82, made up of: feature A contributed +0.31, feature B contributed +0.18, feature C contributed -0.05, ...'. This is the explainability layer required for regulatory compliance and analyst review. We produce both global summary plots (which features matter overall) and per-account explanations stored in account_explanations.json.

Phase 6 — Governance artifacts

reports/model_card.md

A one-page document required by the RBI's draft FREE-AI framework for any ML system used in customer-impacting decisions. It documents intended use, training data summary, performance per demographic segment (account type, region, gender, occupation), known limitations, ethical considerations, and conditions that would trigger retraining.

reports/data_audit.md

The leakage audit report. Lists every feature with high correlation to F2230 or the target, with a one-line justification for whether it was kept or dropped.

reports/feature_semantics.md

For each of the top 20 features by SHAP importance, a plausible banking interpretation. Example: 'F2122 — high values strongly predict mule. Distribution skewed right with positives in the upper tail. Plausibly a velocity or counterparty-count feature.' We cannot be certain because the features are anonymized, but the word 'plausibly' does the work — we use the statistical behaviour to infer likely meaning, which is the same inference a banking analyst would do without a data dictionary.

8. The model results — what the numbers mean

This section explains every headline number from the model, what the metric measures, and why our number is good or bad.

Headline results

Metric	Value	Interpretation
Precision @ top-1% alerts	0.80	Of the riskiest 1% of accounts we flag, 80% are real mules
PR-AUC (95% CI)	0.87 [0.74, 0.97]	Strong precision-recall trade-off; CI is wide due to small positive count
Cross-validation PR-AUC	0.781 ± 0.087	Stable across 15 different data splits
ROC-AUC	0.96	Excellent ranking ability
Brier score	0.0024	Probability outputs are well-calibrated

Why these specific metrics

Precision @ top-1% alerts

This is our headline operational metric. The reasoning: banks have a fixed number of fraud analysts who can review flagged accounts. If we flag the top 1% of accounts as 'check these,' that means about 90 accounts per day from this dataset's scale. Precision @ top-1% asks: of those 90, how many are real mules? Our number is 80%, meaning 72 real mules and 18 false positives per 90 reviewed. That is a workable workload.

PR-AUC (Precision-Recall Area Under Curve)

Standard metric for imbalanced classification. It measures the trade-off between precision (of flagged accounts, how many are real) and recall (of real mules, how many we catch) across all possible thresholds. A perfect model has PR-AUC = 1.0. A random model on our data would have PR-AUC ≈ 0.009 (the base rate). Our 0.87 is very strong.

Bootstrap 95% confidence interval

Because we only have 24 positives in the test set, any single PR-AUC number is noisy. We resample the test set 1000 times with replacement and report the 5th-to-95th percentile range. The interval [0.74, 0.97] tells us the true performance is very likely in that range. Reporting the CI is honest — it acknowledges uncertainty rather than hiding it behind a single number.

ROC-AUC

Measures how well the model ranks positives above negatives. Less useful than PR-AUC for imbalanced data because it can look misleadingly good, but we report it because it is the most familiar metric. Our 0.96 is excellent.

Brier score

Measures calibration quality — how well predicted probabilities match actual probabilities. Lower is better. 0.0024 means when the model says '70% chance of being a mule,' that group of accounts really does contain about 70% mules. Calibration matters because risk bands (Low/Medium/High) need probability cutoffs to mean what they say.

Risk band breakdown

Calibrated probabilities are bucketed into three operational bands. Here is what the test set looked like through this lens:

Band	Score range	Test accounts	True mules in band	Precision
High	≥ 0.70	15	15	100%
Medium	0.30 – 0.70	3	3	100%
Low	< 0.30	2,503	6	0.24%

18 of 24 test mules (75%) land in actionable bands at 100% precision. The 6 mules in the Low band are the documented recall gap — accounts that look behaviourally normal and the model cannot separate them from genuine legitimate accounts. The mitigation for these is rules-based hard filters (I4C ticket match, device-sharing detection, dormancy-break monitoring), which we describe in the architecture section.

The leakage audit — what fired and why we are fine

Our leakage audit compared LightGBM-on-18-features against LightGBM-on-385-features. The PR-AUC gap is 0.51, which is far above our 0.15 warning threshold. The audit fired. We then investigated.

Three checks confirmed the gap is real signal, not leakage:

22. Feature importance shows a smooth monotonic decline across the top 50 features — no single feature dominates as it would if it were a leakage proxy.
23. The top 10 features have max correlation of 0.063 with the non-October-rows indicator — far below any leakage threshold (we use 0.95).

24. Two of the 18 bank-hinted features (F670, F2082) failed bootstrap stability — meaning the bank's hint set is not perfect either, and the gap partly reflects the bank's own incomplete intuition rather than us cheating.

Conclusion documented in reports/model_results.md: gap is honest, audit cleared.

9. Architecture: the system around the model

The model is the brain. The full solution is the body around it — the systems that get data in, score it, decide what to do, take action, log everything, and learn from feedback. This section describes the architecture we are building.

The pipeline, layer by layer

Layer 1 — Sources

All the channels and feeds the system has to consume: UPI, IMPS, NEFT/RTGS, Cards, AePS, Net Banking, Branch CBS transactions, plus FMS alerts, TMS alerts, I4C/CFCFRMS tickets, RBI advisories, and FIU-IND watchlists. For the hackathon demo we simulate this by replaying the test set as if it were a live stream.

Layer 2 — Ingestion

In production this would be Apache Kafka topics — one per source — receiving messages in real time. Schema-registry enforced. For the hackathon we use an HTTP loop posting accounts to the scoring API at a fixed rate. Redpanda (a Kafka-compatible single-binary tool) is a stretch goal if time permits.

Layer 3 — Stream processing

Normalize every source to a canonical schema. Enrich with KYC profile, device fingerprint, geo. Compute online windowed features (transactions per hour, unique counterparties per hour, fan-in/fan-out over 24h). For the hackathon, the static dataset already contains pre-computed versions of these features so we do not need to build the stream-processing layer.

Layer 4 — Scoring service

A FastAPI application. Loads the trained LightGBM model. Exposes endpoints:

- POST /score — takes account features, returns score, band, top reasons, matched rules, model version
- GET /accounts — paginated list of accounts for the alert queue
- GET /accounts/{id} — full detail for one account with SHAP explanation
- POST /feedback — analyst disposition writes to retrain queue
- POST /tickets/i4c — adds an account to the hard-block list

Every score response is written to logs/decisions.jsonl as an append-only audit log — this is our WORM (Write Once, Read Many) compliance artifact.

Layer 5 — Decision engine

A separate policy module. Takes (score, matched_rules, account_state) and returns an action. The policy matrix:

Risk band	Account state	Action
Low (<0.30)	Any	ALLOW, log only
Medium (0.30–0.70)	New / low-history	STEP-UP authentication (OTP + device check)
Medium	Established	ALLOW with soft alert to TMS queue
High (≥0.70)	Any	HOLD outgoing leg, route to analyst, 15-min SLA
Hard hit (I4C/CFCFRMS match)	Any	FREEZE credit, draft STR, notify nodal officer

Decision policy is deliberately separate from the model. This means policy can be tightened or relaxed without retraining — when a new fraud pattern emerges and the bank wants to be more aggressive, they change the cutoffs, not the model.

Layer 6 — Action and feedback

Downstream systems consume decisions: core banking (acts on freezes), case management (gets analyst queue), the analyst UI (shows alerts), FIU-IND filing (gets STR drafts), and the audit log (records everything). Analyst dispositions flow back as feedback. The feedback queue becomes training data for the next retrain cycle.

The closed feedback loop

This is the single most important production property of the system. When an analyst marks an account as a false positive, that decision gets written to data/feedback/queue.jsonl. The retraining pipeline (monthly cron, or triggered by drift detection) consumes the queue, adds correctly-labelled rows to the training set, retrains the model, and compares against the current production model (champion/challenger). If the challenger wins, it gets promoted. Every prediction is tagged with the model version that made it, so we can audit which model handled which transaction.

Hackathon scope decisions

A full production system takes months to build. For the hackathon submission we focus on what is demoable end-to-end.

Must-have (will be in the demo)

- Trained model with reported metrics and calibration
- FastAPI scoring service with all endpoints functional
- Decision engine implementing the full policy matrix
- Next.js analyst console showing live alerts with SHAP reasons

- Visible closed feedback loop — mark false positive, see retrain queue update
- Simulated I4C ticket triggering auto-FREEZE
- Synthetic graph view of a 'mule ring'
- Latency benchmark proving sub-100ms p95
- One-page Model Card
- Management dashboard with KPI charts

Stretch (will be mentioned as roadmap)

- Redpanda (Kafka) for actual streaming ingestion
- STR auto-draft PDF generation for FIU-IND filing
- Neo4j graph backend for real transfer-edge analysis
- Airflow DAG for scheduled retraining

10. Where we are now and what comes next

What is complete

- Phase 0 — repository scaffold, environment, dependencies
- Phase 1 — full EDA, problems discovered before modelling
- Phase 2 — preprocessing module with dedup, split, and leakage audit
- Phase 3 — feature engineering and bootstrap stability selection (2,249 → 385 features)
- Phase 4 — model training across five candidates, leakage audit fired and cleared
- Phase 5 — variance estimation, isotonic calibration, SHAP explanations
- Phase 6 — model card, data audit report, feature semantics report

Headline numbers (locked)

- Precision @ top-1% alerts: 0.80
- PR-AUC: 0.87 [0.74, 0.97]
- CV PR-AUC: 0.781 ± 0.087
- ROC-AUC: 0.96
- Brier score: 0.0024
- Latency target: <100 ms p95 (to be measured)

The 15-day plan to submission

Submission deadline is 5 June 2026. We have 15 working days. The model is frozen — no more changes to scoring logic. Remaining work is the demo surface, integration, and presentation.

Day(s)	Focus	Deliverable
May 21	FastAPI service + benchmark + WORM log	Backend locked
May 22	Next.js scaffold + design system + 3-panel layout	Bare UI skeleton
May 23	Wire data layer, React Query hooks	Real predictions in UI
May 24	SHAP explanation panel, risk gauge, policy badge	Account detail page complete
May 25	Action buttons + feedback loop with visible state	Closed-loop demo works
May 26	Snapshot replay + I4C ticket auto-FREEZE demo beat	Live-feeling stream

Day(s)	Focus	Deliverable
May 27	Mule ring graph visualization (React Flow)	Graph view shipped
May 28	Management dashboard route with KPIs	Executive view shipped
May 29	Stretch: Redpanda streaming	Optional Kafka beat
May 30	Stretch: STR auto-draft PDF	Optional compliance beat
May 31	Polish pass — empty states, loading, mobile	Submission-ready visuals
Jun 1	Final report PDF + 12-slide deck	Documents shipped
Jun 2	Demo video recording + first rehearsal	Backup video ready
Jun 3	Two more rehearsals + fix list	Demo bulletproof
Jun 4	Submit by evening	Done

Risk register

Risk	Mitigation
Frontend takes longer than planned	Drop the stretch items (Redpanda, STR draft) before cutting must-haves
Live demo fails on submission day	Pre-recorded 3-minute video uploaded by day 13 as backup
Temptation to retrain model with one more tweak	Model is frozen Day 1. No exceptions, even if a metric looks improvable
Submission portal closes early or has bugs	Submit Day 15 evening (June 4), not Day 16
Judge asks about features we don't understand	feature_semantics.md documents plausible interpretations for top 20
Judge questions the false-negative gap	Documented in model card §6 with rules-based mitigation story

11. Glossary of every term used

Every term in alphabetical order. If you encounter unfamiliar language anywhere in the project documentation, look here first.

AePS — Aadhaar-enabled Payment System. Cash withdrawal at agent points using fingerprint authentication.

AUC — Area Under Curve. Generic term for the area under a precision-recall or ROC curve. Higher is better.

Bootstrap — Statistical technique: resample your data with replacement many times and aggregate results. Used to estimate uncertainty and to test stability of feature selection.

Brier score — Measures how well predicted probabilities match observed outcomes. Lower is better. Used to assess calibration.

Calibration — The property that predicted probabilities match real-world frequencies. A well-calibrated model that says '70% chance' produces accounts that really are mules 70% of the time.

CBS — Core Banking System. The bank's central ledger software.

CDD — Customer Due Diligence. Ongoing checks a bank performs on customer activity beyond initial KYC.

CFCFRMS — Citizen Financial Cyber Fraud Reporting and Management System. I4C's backend used to route reported frauds to banks.

Champion/challenger — Production pattern where a new (challenger) model is tested against the current (champion) before being promoted.

Class imbalance — When one class is much rarer than the other in classification problems. Mule detection has ~1% positive rate.

CV (cross-validation) — Splitting training data into folds, training on some and validating on others, rotating which is which. Gives a more robust performance estimate.

Dedup — Deduplication. Identifying columns that contain identical values and keeping one representative.

Drift — When the statistical properties of incoming data change over time, making the model less accurate.

FastAPI — Python web framework used to build the scoring service. Fast, modern, has automatic documentation.

Feature — An input variable to a model. In our dataset, each of F1 through F3924 is a feature.

FIU-IND — Financial Intelligence Unit, India. Receives suspicious transaction reports and coordinates with police.

FMS — Fraud Monitoring System. Existing rule-based software a bank runs to flag suspicious transactions.

FREE-AI — Framework for Responsible and Ethical Enablement of AI. RBI's draft guidance for AI in Indian banking.

Holdout — A portion of data deliberately set aside and never used during training, reserved for final evaluation.

Hyperparameter — A model setting chosen by the engineer (like 'maximum tree depth'), as opposed to weights the model learns from data.

I4C — Indian Cyber Crime Coordination Centre. Federal body coordinating cyber-crime response.

IMPS — Immediate Payment Service. 24×7 instant transfer between bank accounts.

Isotonic regression — A simple, monotone way to map raw model outputs to true probabilities. Used for calibration.

KYC — Know Your Customer. Identity verification performed when opening an account.

Leakage — When a feature accidentally contains information about the target that would not be available at prediction time. Causes overconfident models that fail in production.

LightGBM — Gradient boosting library. Our primary model. Handles missing values natively, fast on wide data.

Mule account — A bank account used to receive and forward illicit funds, where the holder may be unwitting, paid, or fully fraudulent.

NCRP — National Cybercrime Reporting Portal. Citizen-facing website where fraud victims file complaints.

NEFT — National Electronic Funds Transfer. Batched bank-to-bank transfer.

Next.js — React framework used to build the analyst console UI.

PR-AUC — Precision-Recall Area Under Curve. Primary evaluation metric for imbalanced classification.

Precision — Of the items we flagged, what fraction were correct. High precision = few false positives.

Precision @ top-K — Operational metric: of the top K accounts by score, what fraction are real positives. Matches the reality of fixed analyst capacity.

RBI — Reserve Bank of India. The central bank and primary regulator.

Recall — Of all the real positives, what fraction did we catch. High recall = few false negatives.

Redpanda — Kafka-compatible streaming platform. Single binary, easier than Kafka for hackathons.

RTGS — Real-Time Gross Settlement. Instant transfer for amounts above ₹2 lakh.

ROC-AUC — Receiver Operating Characteristic Area Under Curve. Measures ranking quality but can mislead on imbalanced data.

scale_pos_weight — LightGBM parameter telling the model to weight the rare positive class more heavily. We set it to ~111 (the negative-to-positive ratio).

SHAP — SHapley Additive exPlanations. Method for explaining individual predictions by attributing contributions to each feature.

SLA — Service Level Agreement. The promised time within which a flagged account will be reviewed.

SMOTE — Synthetic Minority Oversampling Technique. Generates synthetic positive examples. We deliberately do not use it because with 57 training positives, synthetic data would be statistical fiction.

Stability selection — Running feature selection many times on bootstrap subsamples and keeping only features that survive consistently.

Stratified split — Train/test split that preserves the proportion of each class in both halves.

STR — Suspicious Transaction Report. Formal report filed with FIU-IND.

Streamlit — Python framework for quickly building data app UIs. Considered for our UI but rejected in favour of Next.js given the time available.

TMS — Transaction Monitoring System. Existing rule-based software flagging suspicious transaction patterns.

UPI — Unified Payments Interface. India's instant payment system using phone-linked virtual addresses.

WORM — Write Once, Read Many. Audit-log property where entries cannot be modified after writing. Required for compliance.

— end of document —